

Neural Operator Poisson Safety Functions for Robust Model-Free Safety-Critical Control

Anonymous authors

Abstract—Autonomous robotic systems require safety-critical control that can quickly respond to complex and changing environments. In this paper, we propose a robust model-free safety-critical control framework that integrates a neural operator with Poisson-based safety functions (PSFs) for real-time collision avoidance. The neural operator enables fast inference of a PSF without solving a partial differential equation (PDE) repeatedly, mitigating the computational burden. However, approximation of the PDE solution introduces errors in both the predicted safety function and its derivatives; thus, we derive a worst-case robust safety condition under bounded approximation errors. Since online computation of worst-case error bounds is challenging in changing environments, we further propose a practical online error bound estimation that leverages obstacle-boundary measurements from onboard sensors (e.g. LiDAR). Lastly, we validate the proposed framework through simulations and experiments on a mobile robot, showing that the Fourier neural operator-based controller achieves efficient collision avoidance while providing accurate real-time PSF prediction. The project page is available at <https://npsf-cbf.github.io>.

I. INTRODUCTION

Safe robot control in complex and dynamic environments requires an efficient collision avoidance framework that can react to moving obstacles and be applied across different robotic systems. Control Barrier Functions (CBFs) provide a useful tool for collision avoidance by rendering a safe set forward invariant [1], and recent model-free safety-critical control [2] further facilitates their practical deployment without requiring explicit knowledge of the robot dynamics under a stable low-level velocity controller. However, the practical deployment still relies on constructing suitable safety functions that represent complex and dynamic obstacle geometries. Poisson-based safety functions (PSFs) provide a smooth safety function that can be directly used as a control barrier function for safe robot control [3], but updating them in changing environments requires repeatedly solving a Poisson partial differential equation (PDE); this results in a computational bottleneck at high-dimensional spatial resolutions. In this paper, we aim to tackle this challenge by introducing a neural operator to approximate the solution operator of the Poisson PDE and consequently predict a PSF across varying environments. Neural operators [4] provide a promising alternative to repeated numerical PDE solves, learning a mapping between function spaces and then approximating a PDE solution operator. This operator-learning nature can be used to rapidly construct a PSF from changing environments, substantially reducing the computational burden. In spite of computational benefits, such an approach comes with a critical safety challenge; replacing the exact numerical PDE

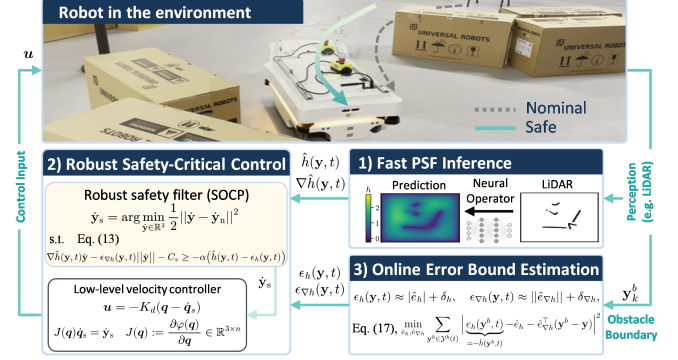


Fig. 1. Overview of the proposed robust model-free safety-critical control framework that integrates a neural operator to infer a Poisson safety function.

solver with a neural operator can introduce an approximation error in both the predicted PSF and its derivatives, potentially compromising the safety guarantees. Therefore, our approach addresses two key aspects that have not yet been considered simultaneously: first, efficient reconstruction of a PSF by a neural operator and second, ensuring robust safety guarantees under neural approximation errors.

Contributions. We develop a robust model-free safety-critical control framework that leverages a neural operator to rapidly infer a PSF and integrate it into the forward invariance condition as shown in Fig. 1.

1) Fast Poisson safety function (PSF) inference: We use a neural operator to learn a solution operator for the Poisson PDE, allowing fast PSF construction across varying environments without repeatedly solving computationally expensive PDEs.

2) Robust model-free safety-critical control: We derive a robust model-free safety-critical control design that explicitly accounts for bounded approximation errors in the inferred safety function and its derivatives, providing worst-case safety guarantees without requiring explicit knowledge of the robot dynamics.

3) Online error bound estimation: In practice, obtaining worst-case approximation error bounds online is non-trivial; thus we propose local approximation error estimation that exploits measurements of obstacle boundaries from onboard perception sensors (e.g. LiDAR), which are then used as robust margins for practical deployment in the real world.

4) Experimental validation: Lastly, we evaluate the proposed framework in collision avoidance scenarios through both simulations and experiments, demonstrating substantial computational gains beyond a numerical PDE solver while maintaining safety in complex and changing environments.

II. RELATED WORK

Several works have been introduced around safety-critical control, constructing safety functions, and neural operators to approximate the solutions of partial differential equations.

A. Safety-Critical Control

Formal safety guarantees can be achieved by enforcing forward invariance condition via CBFs [1]. However, the key challenge of using CBFs is that safety might be violated when model uncertainty is introduced in the system. To remedy this, diverse types of uncertainties have been considered in designing CBFs-based safe controllers, such as parametric uncertainty [5] and state uncertainty [6]. Recently, to increase practical use cases and mitigate model dependency, model-free safety-critical control [2] has been getting attention. Such an approach does not require explicit knowledge of complicated robotic systems, but ensures formal safety guarantees under a stable low-level velocity controller, which is a well-established implementation in robotics [7]. Despite recent advances toward safety-critical controllers, constructing suitable safety functions still remains a key challenge for complex environments.

B. Safety Function Representations

Artificial potential fields (APFs) [8] have been used to construct safety functions in spatial environments, and they are efficiently incorporated with the CBFs-based formulation while reducing extensive parameter tuning and undesirable oscillations [9]. Another geometrically intuitive tool is signed distance functions (SDFs), however, they are not necessarily continuously differentiable and may lead to ill-conditioned optimization problems when integrated into CBF-based controllers [10]. To resolve this limitation, a prior work has leveraged smooth approximations of SDFs to accomplish CBFs-based control synthesis [11]. Nevertheless, SDFs-based approaches typically require an individual safety function for each environmental obstacle. In contrast, recent works have used a PSF which constructs a global safety field from an environmental occupancy map by solving Poisson's PDE with a boundary condition [3] via a numerical solver, successive over relaxation (SOR) [12] and subsequently integrated it with CBFs-based controllers [13], [14]. However, iteratively solving the PDE at each time step in cluttered environments demands substantial computation resources, as the spatial resolutions become finer.

C. Neural Operator

Neural operators are designed to learn a mapping between function spaces, enabling efficient approximation of PDE solution operators [4], in contrast to conventional neural network architectures that commonly learn mappings between vector spaces. Recent network architectures of neural operators include deep operator network (DeepONet) [15] using branch-trunk network, and Fourier neural operator (FNO) that is based on a learnable Fourier integral operator [16]. For safety-critical control systems, a neural operator has been incorporated into safe PDE boundary control to

maintain the boundary output of a PDE within a safe set [17]. In another example, a neural operator is employed to learn the solution operator of a Hamilton-Jacobi PDE which maps environmental parameters to a corresponding CBF [18]. However, this work still relies on an accurate system model, which may lead to potential safety violations when there exists model mismatch.

III. PROBLEM STATEMENT

Let $\mathcal{Q}$ be an n -dimensional configuration space and $\mathcal{U} \subseteq \mathbb{R}^m$ be the admissible input space. Consider the fully-actuated robotic system:

$$D(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{u}, \quad (1)$$

where $\mathbf{q} \in \mathcal{Q}$ represents the generalized coordinates, and $\mathbf{u} \in \mathcal{U}$ denotes the control input, and $D(\mathbf{q}) \in \mathbb{R}^{n \times n}$, $C(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{n \times n}$, $\mathbf{g}(\mathbf{q}) \in \mathbb{R}^n$ represent the mass matrix, the centrifugal and Coriolis matrix, and the gravity vector, respectively.

In this work, we focus on model-free safety control at the kinematic level (e.g. collision avoidance in task-space). Since environmental safety is defined in the task space while the robot dynamics evolve in the configuration space, we first introduce a kinematic mapping between the two representations. Let $\varphi : \mathcal{Q} \rightarrow \mathbb{R}^3$ be a kinematic mapping function from the configuration space to the task space, i.e. $\mathbf{y} = \varphi(\mathbf{q})$ where $\mathbf{y} \in \mathbb{R}^3$ denotes the task-space position. We then define the zero-superlevel time-varying safe set of a continuously differentiable function $h : \mathbb{R}^3 \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$:

$$\mathcal{C}_{\mathcal{T}}(t) = \{\mathbf{y} \in \mathbb{R}^3 \mid h(\mathbf{y}, t) \geq 0\}. \quad (2)$$

In practice, constructing $h(\mathbf{y}, t)$ is nontrivial, especially for changing environments with complex geometries. Poisson-based safety functions provide a useful tool to construct a smooth safety field directly from environmental occupancy information [3]. Let $\Omega(t) \subset \mathbb{R}^3$ denote the open, bounded, connected set with smooth boundary, $\partial\Omega(t)$. A Poisson safety function is obtained at every time step by solving the Poisson partial differential equation (PDE) with a Dirichlet boundary condition:

$$\begin{cases} \Delta h(\mathbf{y}, t) = f(\mathbf{y}, t), & \text{in } \Omega(t), \\ h(\mathbf{y}, t) = 0, & \text{on } \partial\Omega(t), \end{cases} \quad (3)$$

where Δ is the Laplacian, and $f : \Omega \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$ is the forcing term. As the environment changes, the Poisson PDE (3) should be repeatedly solved at each time step, leading to a substantial computation burden at higher spatial resolutions, e.g., with N^2 scaling for a two-dimensional grid [3]. To alleviate this, we consider a learning-based approach to achieve fast inference, in which a neural operator approximates the Poisson solution operator to predict the PSF. In spite of the fast inference, approximation errors in the predicted PSF can invalidate safety guarantees unless they are explicitly accounted for, motivating the need for a robust safety control design. Accordingly, in this paper, we develop a robust model-free safety-critical control framework using the Poisson-based safety function inferred by a neural operator.

IV. BACKGROUND

This section revisits model-free safety-critical control, Poisson safety functions, and neural operators.

A. Model-Free Safety-Critical Control

Instead of relying on an accurate model of the system (1), the model-free safety-critical control scheme enables safe robot control from a kinematic perspective through a stable low-level velocity controller. For model-free safety-critical control, we hold the following assumptions:

Assumption 1: Given a reference velocity, $\dot{\mathbf{q}}_d \in \mathbb{R}^n$, there exists an exponentially stable velocity tracking controller $\mathbf{u} = k(\mathbf{q}, \dot{\mathbf{q}})$, satisfying $\|\dot{\mathbf{e}}(t)\| \leq M\|\dot{\mathbf{e}}_0\|e^{-\lambda t}$ for some $M, \lambda > 0$ where $\dot{\mathbf{e}} = \dot{\mathbf{q}} - \dot{\mathbf{q}}_d$ and $\dot{\mathbf{e}}_0 = \dot{\mathbf{e}}(0)$ [2].

Assumption 2: Given a safe set in task-space, $\mathcal{C}_{\mathcal{T}}(t)$ for the system (1), we assume that $\|\frac{\partial h(\mathbf{y}, t)}{\partial \mathbf{y}}\| \leq C_h, \|\frac{\partial h(\mathbf{y}, t)}{\partial t}\| \leq C_v$ where $C_h, C_v \in \mathbb{R}_{>0}$ for $\forall \mathbf{y} \in \mathcal{C}_{\mathcal{T}}(t)$.

Consider the system (1) and a safe set from (2), and let *Assumptions 1 and 2* hold. Then, the safety in task-space is ensured for the system through the following theorem.

Theorem 1 ([2], [14]): Given the safe set, $\mathcal{C}_{\mathcal{T}}(t)$, and assume that $\mathbf{y} \in \mathcal{C}_{\mathcal{T}}(0)$ and $\dot{\mathbf{e}}_0 = \mathbf{0}$. Let $\dot{\mathbf{y}}_s \in \mathbb{R}^3$ be the safe task-space velocity obtained by minimally modifying the nominal task-space velocity $\dot{\mathbf{y}}_n \in \mathbb{R}^3$ as:

$$\begin{aligned} \dot{\mathbf{y}}_s &= \arg \min_{\dot{\mathbf{y}} \in \mathbb{R}^3} \frac{1}{2} \|\dot{\mathbf{y}} - \dot{\mathbf{y}}_n\|^2 \\ \text{s.t. } \nabla h(\mathbf{y}, t)^\top \dot{\mathbf{y}} + \frac{\partial h}{\partial t}(\mathbf{y}, t) &\geq -\alpha(h(\mathbf{y}, t)), \end{aligned} \quad (4)$$

where $\alpha \in \mathcal{K}_\infty^e$. In particular, α may be selected as the linear function, $\alpha(h) = \gamma h$ such that $\lambda > \gamma > 0$. If there exists a controller $\mathbf{u}$ that exponentially tracks a corresponding safe velocity in configuration space $\dot{\mathbf{q}}_s$ such that $J(\mathbf{q})\dot{\mathbf{q}}_s = \dot{\mathbf{y}}_s$, where $J(\mathbf{q}) := \frac{\partial \varphi(\mathbf{q})}{\partial \mathbf{q}} \in \mathbb{R}^{3 \times n}$, then the robotic system (1) is safe with respect to $\mathcal{C}_{\mathcal{T}}(t)$.

B. Poisson Safety Functions for Collision Avoidance

For safe robot navigation, a smooth function should be provided to characterize a safe set. One promising approach is that we can use the Poisson PDE [3] to obtain the smooth function from an occupancy map by solving the Dirichlet problem defined in (3). In practice, environmental occupancy information can be obtained via perception and subsequently converted into a smooth safety function by solving (3) repeatedly. The solution of the PDE, h is a CBF, provided that the forcing term, f is smooth on the domain, $C^{k,\beta}(\bar{\Omega}(t), \mathbb{R}_{<0})$ for some $k \in \mathbb{N}_0, \beta \in (0, 1)$ where $\bar{\Omega}(t) = \Omega(t) \cup \partial\Omega(t)$ [3]. The potential forcing term is a Hölder continuous function, $f \in C^{0,\beta}(\bar{\Omega}(t); \mathbb{R}_{<0})$ defined as :

$$f(\mathbf{y}, t) = - \left(\frac{\text{dist}(\mathbf{y}, \partial\Omega(t))}{\|\text{dist}(\mathbf{y}, \partial\Omega(t))\|_\infty} \right)^\beta, \quad \forall \mathbf{y} \in \bar{\Omega}(t), \quad (5)$$

where $\text{dist}(\mathbf{y}, \partial\Omega(t)) = \min_{\mathbf{y}_o \in \partial\Omega(t)} \|\mathbf{y} - \mathbf{y}_o\|$. However, computational cost increases significantly as spatial resolution increases. To enable fast construction of a PSF, we leverage

a learning-based approach, a neural operator to learn the solution operator for the Poisson PDE (3).

C. Neural Operators

Neural operators approximate the nonlinear mappings between function spaces and enable solving PDEs efficiently [4]. A nonlocal neural operator is a general tool that holds the universal approximation theorem [19], which covers several types of neural operators such as Deep Operator Network (DeepONet) and Fourier Neural Operator (FNO). Let $\Omega_u \subset \mathbb{R}^{n_u}$ and $\Omega_v \subset \mathbb{R}^{n_v}$ be the bounded Lipschitz domains, and denote their closures by $\bar{\Omega}_u, \bar{\Omega}_v$, respectively. Suppose that there exist continuous function spaces, $C^s(\bar{\Omega}_u; \mathbb{R}^c)$ and $C^{s'}(\bar{\Omega}_v; \mathbb{R}^v)$ where $s, s' \in \mathbb{N}_0$. Then, the approximation error is formally guaranteed by the following theorem:

Theorem 2 (Universal Approximation Theorem [19]):

Let Ψ be a continuous operator mapping $C^s(\bar{\Omega}_u; \mathbb{R}^c) \mapsto C^{s'}(\bar{\Omega}_v; \mathbb{R}^v)$, and let $K \subset C^s(\bar{\Omega}_u; \mathbb{R}^c)$ be a compact set. Then the approximated neural operator, $\hat{\Psi} : K \mapsto C^{s'}(\bar{\Omega}_v; \mathbb{R}^v)$ holds for any $\epsilon > 0$:

$$\sup_{u_o \in K} |\Psi(u_o)(y) - \hat{\Psi}(u_o)(y)| \leq \epsilon, \quad \forall y \in \bar{\Omega}_v. \quad (6)$$

In this paper, we employ two widely used neural operator architectures, DeepONet and FNO, described below.

Definition 1 (Deep Operator Network [15]): A Deep Operator Network (DeepONet) approximates a nonlinear operator, $\mathcal{G}$ using branch and trunk neural networks as follows:

$$\mathcal{G}(u_o)(y) \approx \sum_{k=1}^p \underbrace{b_k(\mathbf{u}_o; \Theta^k)}_{\text{branch}} \underbrace{t_k(y; \theta^k)}_{\text{trunk}}, \quad (7)$$

where $p, m \in \mathbb{N}$. $b_k(\cdot; \Theta^k)$ and $t_k(\cdot; \theta^k)$ denote branch and trunk neural networks, respectively, and the corresponding network parameters are denoted by $\Theta^k \in \mathbb{R}^{d_\Theta}$ and $\theta^k \in \mathbb{R}^{d_\theta}$. The input of b_k , $\mathbf{u}_o$ is the system measurement, defined as $[u_o(x_1), u_o(x_2), \dots, u_o(x_m)]^\top$ where $x_i \in \Omega_u, i = 1, \dots, m$.

Definition 2 (Fourier Neural Operator [16]): A Fourier Neural Operator (FNO) is defined as the composition of L Fourier layers, $\Phi_\ell, \ell = 0, 1, \dots, L-1$, each consisting of a learnable Fourier integral operator $\mathcal{K}_\ell$ and a nonlinear activation function, $\sigma(\cdot)$ in the following form:

$$\mathcal{G}(u_o)(y) \approx [(Q \circ \Phi_{L-1} \circ \dots \circ \Phi_0 \circ P)(u_o)](y), \quad (8)$$

where $\Phi_{\ell+1}(y) := \sigma(W_\ell \Phi_\ell(y) + (\mathcal{K}_\ell \Phi_\ell)(y)), (\mathcal{K}_\ell \Phi_\ell)(y) := \mathcal{F}^{-1}(R_\phi \cdot (\mathcal{F} \Phi_\ell))(y)$, and $\mathcal{F}$ is the Fourier transform and $\mathcal{F}^{-1}$ denotes its inverse transform. R_ϕ is the Fourier transform of a periodic function governed by the learnable parameters, $\phi \in \Theta_{\mathcal{K}}$. W_ℓ is a linear transformation matrix, and P, Q are a lifting layer and a projection layer, respectively.

A solution operator of the Poisson PDE (3), which maps the forcing function (5) to the safety function h , can be approximated by a neural operator. The learned operator enables efficient construction of a PSF in changing environmental conditions while avoiding repeated numerical PDE solves.

V. SAFE CONTROL VIA NEURAL OPERATOR LEARNING

We first describe how to learn safety functions via neural operators and subsequently introduce the proposed framework that incorporates the predicted safety function into model-free safety-critical control, considering worst-case approximation error bounds. Lastly, we present a perception-based online approximation error bound estimation.

A. Learning Safety Function via Neural Operator

Our key insight is that the solution operator of (3) maps the forcing function (5) to the corresponding safety function h , and the operator can be learned using a neural operator; thus we aim to learn the functional mapping between f and h through an operator, $\mathcal{G}$ which is called *PSF operator*. As established in [3], h is determined uniquely by f , and we define the operator as follows:

$$\mathcal{G} : f(\mathbf{y}, t) \mapsto h(\mathbf{y}, t), \quad h(\mathbf{y}, t) = \mathcal{G}(f(\mathbf{y}, t))(\mathbf{y}), \quad (9)$$

for all $\mathbf{y} \in \mathbb{R}^3$. For neural operator training, we first construct input–output training datasets from randomly generated occupancy maps that capture environments with uniformly sampled obstacle configurations. From each occupancy map, we compute the corresponding forcing term, f , and subsequently solve (3) using successive over relaxation (SOR) [12] to obtain the corresponding PSF, h with respect to $\mathbf{y}$. We then construct the dataset, $\mathcal{D} = \{(f_i, h_i, \nabla h_i)\}_i^N$ where N denotes the number of samples. Afterwards, the neural operator (modeled using either DeepONet or FNO) is trained in a supervised manner by minimizing the loss function, $\mathcal{L} = \mathcal{L}_h + \lambda_{\nabla h} \mathcal{L}_{\nabla h}$, where

$$\mathcal{L}_h = \frac{1}{NM} \sum_{i=1}^N \sum_{j=1}^M \left| \underbrace{\hat{\mathcal{G}}_\theta(f_i(\mathbf{y}_j, t))(\mathbf{y}_j)}_{\hat{h}_i(\mathbf{y}_j, t)} - h_i(\mathbf{y}_j, t) \right|^2, \quad (10)$$

$$\mathcal{L}_{\nabla h} = \frac{1}{NM} \sum_{i=1}^N \sum_{j=1}^M \left\| \nabla \hat{h}_i(\mathbf{y}_j, t) - \nabla h_i(\mathbf{y}_j, t) \right\|^2, \quad (11)$$

where $\lambda_{\nabla h} > 0$ is the weight coefficient, and M denotes the number of spatial grid points in each sample. After training, $\hat{\mathcal{G}}_\theta$ predicts $\hat{h}(\mathbf{y}, t)$, and we then compute its spatial gradient, $\nabla \hat{h}(\mathbf{y}, t)$ based on finite differences.

B. Worst-Case Safety for Model-Free Safety-Critical Control

For formal safety guarantees, we cannot just replace h in (4) with the approximated PSF, $\hat{h}$ because of the approximation error between h and $\hat{h}$; thus we consider worst-case safety guarantees and incorporate the approximated PSF into (4) with worst-case error bounds. Following [20, Proposition 1], suppose that there exist point-wise approximation error bounds $e_h(\mathbf{y}, t) \in \mathbb{R}_{\geq 0}$ and $e_{\nabla h}(\mathbf{y}, t) \in \mathbb{R}_{\geq 0}$ such that:

$$|e_h(\mathbf{y}, t)| \leq \epsilon_h(\mathbf{y}, t), \quad \|\mathbf{e}_{\nabla h}(\mathbf{y}, t)\| \leq \epsilon_{\nabla h}(\mathbf{y}, t), \quad (12)$$

where $e_h(\mathbf{y}, t) = h(\mathbf{y}, t) - \hat{h}(\mathbf{y}, t) \in \mathbb{R}$ and $\mathbf{e}_{\nabla h}(\mathbf{y}, t) = \nabla h(\mathbf{y}, t) - \nabla \hat{h}(\mathbf{y}, t) \in \mathbb{R}^3$ are the approximation errors at $\mathbf{y}$ in h and its gradient, ∇h , respectively.

Proposition 1 (Safety Guarantees for neural operator):

Consider the system (1) and the safe set, $\mathcal{C}_T(t)$ from (2), and suppose that there exist approximation error bounds from (12). Let $\hat{\mathcal{G}}_\theta(f(\mathbf{y}, t))(\mathbf{y})$ be the learned neural operator which approximates a PSF, $\hat{h}$. If there exists a stable velocity controller $\mathbf{u} = k(\mathbf{q}, \dot{\mathbf{q}})$ that tracks a safe velocity, $\dot{\mathbf{q}}_s$ such that $\dot{\mathbf{y}} = J(\mathbf{q})\dot{\mathbf{q}}_s$, satisfies:

$$\nabla \hat{h}(\mathbf{y}, t) \dot{\mathbf{y}} - \epsilon_{\nabla h}(\mathbf{y}, t) \|\dot{\mathbf{y}}\| - C_v \geq -\gamma(\hat{h}(\mathbf{y}, t) - \epsilon_h(\mathbf{y}, t)), \quad (13)$$

for all $\mathbf{y}$ and $t \geq 0$, then the controller $\mathbf{u}$ renders the system (1) safe w.r.t. $\mathcal{C}_T(t)$.

Proof: Omitting the dependence on t for brevity, we show that (13) implies the following condition:

$$\nabla h(\mathbf{y}) \dot{\mathbf{y}} + \frac{\partial h(\mathbf{y})}{\partial t} + \gamma h(\mathbf{y}) \geq 0. \quad (14)$$

We add and subtract $\gamma \hat{h}(\mathbf{y})$ and $\nabla \hat{h}(\mathbf{y}) \dot{\mathbf{y}}$ to (14), yielding:

$$\begin{aligned} & \nabla \hat{h}(\mathbf{y}) \dot{\mathbf{y}} + \gamma \hat{h}(\mathbf{y}) + \frac{\partial h(\mathbf{y})}{\partial t} + (\nabla h - \nabla \hat{h}) \dot{\mathbf{y}} + \gamma(h - \hat{h}) \\ & \geq \nabla \hat{h}(\mathbf{y}) \dot{\mathbf{y}} + \gamma \hat{h}(\mathbf{y}) - C_v - \|(\nabla h - \nabla \hat{h}) \dot{\mathbf{y}}\| - \gamma|(h - \hat{h})| \\ & \geq \nabla \hat{h}(\mathbf{y}) \dot{\mathbf{y}} - \epsilon_{\nabla h}(\mathbf{y}) \|\dot{\mathbf{y}}\| - C_v + \gamma(\hat{h}(\mathbf{y}) - \epsilon_h(\mathbf{y})) \geq 0. \end{aligned}$$

The first inequality follows from the Cauchy–Schwarz inequality, while the second follows from the upper bounds in (12) and *Assumption 2*. The last inequality is equivalent to (13), which implies (14). This implies that $\mathcal{C}_T(t)$ is forward invariant and the system (1) is safe with respect to $\mathcal{C}_T(t)$. ■

Remark 1: The optimization problem with the safety constraint, (13) can be efficiently solved via second-order cone programming (SOCP) [20]. The bound C_v in *Assumption 2* characterizes the maximum temporal changes in the environment. This assumption excludes abrupt obstacle motions where $\partial h / \partial t$ becomes unbounded.

Example 1 (Double-Integrator System): We demonstrate worst-case safety guarantees on a robot navigation example. Consider the following double-integrator system,

$$\ddot{\mathbf{q}} = \mathbf{u}, \quad (15)$$

where $\mathbf{q} = (x, y)^\top$ is the robot's coordinates in the $x - y$ plane. The objective is to reach the desired goal position, $\mathbf{q}_d = (x_d, y_d)^\top$ from the initial position, $\mathbf{q}_0 = (x_0, y_0)^\top$, while avoiding dynamic obstacles as shown in Fig. 2(d). Similar to [2], we first define the nominal velocity, $\dot{\mathbf{q}}_d = -K_p(\mathbf{q} - \mathbf{q}_d)$ where $K_p > 0$. Since the configuration space coincides with the task space in this example, i.e., $\mathbf{y} = \mathbf{q}$, the safe velocity command, $\dot{\mathbf{y}}_s = \dot{\mathbf{q}}_s$ is computed by minimally modifying the nominal velocity subject to the safety condition in (13). The resulting safe velocity command is then tracked using a proportional velocity tracking controller, $\mathbf{u} = -K_d(\dot{\mathbf{q}} - \dot{\mathbf{q}}_s)$ with a gain, $K_d > 0$. To learn an operator, we generate 1000 occupancy maps by randomly placing three obstacles (e.g. circle, square, and L-shape) and compute the forcing function over the map. We then construct a dataset by solving (3) and use the resulting dataset to train the PSF operators using FNO and DeepONet. For safety,

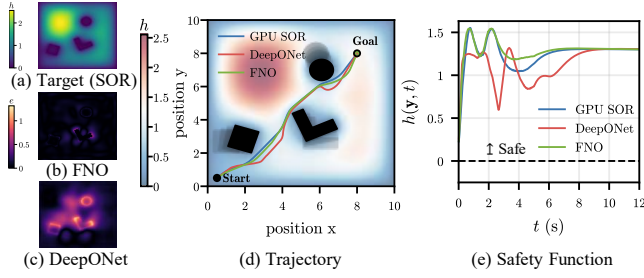


Fig. 2. Numerical simulation of robot navigation in a changing environment. (a) Snapshot of the target h computed using a GPU-accelerated numerical solver, GPU-SOR. (b)–(c) Prediction errors of FNO and DeepONet, corresponding to the target. (d) Robot trajectory over 12s among multiple moving obstacles. (e) Actual safety function values for each method, where $h \geq 0$ indicates non-collision. We conduct the simulation on a 400^2 grid map and observe that the computation times of FNO, DeepONet, and GPU-SOR are 0.68 ms, 1.95 ms, and 6.15 ms, respectively.

we use worst-case robust margins determined based on the approximation errors of each model. Although FNO and DeepONet have approximation errors as shown in Fig. 2(b) and (c), the proposed robust safety control (13) maintains the robot within the safe set while navigating among moving obstacles, as shown in Fig. 2(d) and (e).

Estimating explicit worst-case error bounds online is non-trivial in practice because the actual approximation errors are not directly available without solving a numerical PDE at every time step. This motivates a perception-based online error bound estimation that provides practical error bounds for robot control using onboard sensing.

C. Online Error Bound Estimation

To facilitate practical deployment, we introduce online estimates of the approximation error bounds, $\epsilon_h(\mathbf{y}, t)$ and $\epsilon_{\nabla h}(\mathbf{y}, t)$, using obstacle boundary information from onboard perception. Assuming that we have accurate obstacle boundary measurements from sensing (e.g. LiDAR), the true approximation error at a measured boundary point, $\mathbf{y}^b \in \mathbb{R}^3$ is defined as $e_h(\mathbf{y}^b, t) = -\hat{h}(\mathbf{y}^b, t)$ where $h(\mathbf{y}^b, t) = 0$, because the actual safety function value always satisfies the zero Dirichlet boundary condition on the obstacle boundary from (3). Inspired by [21], at each boundary point of obstacles, $e_h(\mathbf{y}^b, t)$ can be approximated via a first-order Taylor expansion around the current robot position $\mathbf{y}$:

$$e_h(\mathbf{y}^b, t) \approx \hat{e}_h(\mathbf{y}, t) + \hat{e}_{\nabla h}(\mathbf{y}, t)^\top (\mathbf{y}^b - \mathbf{y}), \quad (16)$$

where $\hat{e}_h(\mathbf{y}, t)$ and $\hat{e}_{\nabla h}(\mathbf{y}, t)$ are estimates of the unknown actual errors $e_h(\mathbf{y}, t)$ and $e_{\nabla h}(\mathbf{y}, t)$, respectively. For the sake of the brevity, we replace $\hat{e}_h(\mathbf{y}, t)$ and $\hat{e}_{\nabla h}(\mathbf{y}, t)$ with $\hat{e}_h$ and $\hat{e}_{\nabla h}$, respectively. Given the set of the multiple measured obstacle boundary points, $\mathcal{Y}^b(t) \subset \mathbb{R}^3$, $\hat{e}_h$ and $\hat{e}_{\nabla h}$ are estimated via a least-squares regression in the following:

$$\min_{\hat{e}_h, \hat{e}_{\nabla h}} \sum_{\mathbf{y}^b \in \mathcal{Y}^b(t)} \left| \underbrace{e_h(\mathbf{y}^b, t)}_{=-\hat{h}(\mathbf{y}^b, t)} - \hat{e}_h - \hat{e}_{\nabla h}^\top (\mathbf{y}^b - \mathbf{y}) \right|^2, \quad (17)$$

which yields the local estimates of actual errors at $\mathbf{y}$, based on the measured obstacle boundary. However, $\hat{e}_h$ and $\hat{e}_{\nabla h}$ remain subject to uncertainty since they are estimated only

from the measured obstacle boundary and from a first-order local regression, respectively; thus we introduce uncertainty margins, $\delta_h \in \mathbb{R}_{\geq 0}$ and $\delta_{\nabla h} \in \mathbb{R}_{\geq 0}$ for $\hat{e}_h$ and $\hat{e}_{\nabla h}$. Consequently, the proposed online error bounds are:

$$\epsilon_h(\mathbf{y}, t) \approx |\hat{e}_h| + \delta_h, \quad \epsilon_{\nabla h}(\mathbf{y}, t) \approx \|\hat{e}_{\nabla h}\| + \delta_{\nabla h}, \quad (18)$$

and we use these estimates in (13) for practical deployment. Although the proposed error bound estimation does not establish global error bounds from (12), it provides perception-based robustness margins, enabling collision avoidance in practice. Alternatively, $\epsilon_h, \epsilon_{\nabla h}$ can be estimated empirically using a sample-based approach that compares predictions with the collected ground truth data [20]. However, such empirical estimates may not remain valid in different environments from those represented in the collected data.

VI. SIMULATION

In this section, we present the training of a neural operator for a PSF and evaluate its approximation accuracy and generalization capability. Afterwards, we demonstrate the proposed framework on the mobile robot and quadrotor systems, evaluating computational efficiency of each method.

We aim to train $\hat{G}_\theta$ to predict h . To this end, we first construct a dataset, $\mathcal{D} = \{(f_i, h_i, \nabla h_i)\}_i^N$ by generating 1000 occupancy maps. For each map, we sample the poses of five obstacle geometries from uniform distributions: circle, square, rectangle, L-shape, and triangle as shown in the top-left occupancy map in Fig. 3. We then compute the forcing function from (5) over the map and solve the PDE in (3) to obtain h , using a GPU-parallelized checkerboard SOR solver (GPU-SOR) with a tolerance 10^{-6} [3]. The solution, h and its spatial gradient, ∇h over the entire grid are used as training targets corresponding to the input forcing function.

A. Learning Neural Operator for Poisson Safety Function

To facilitate model training, the values of $h(\mathbf{y}, t)$ are normalized using the mean and standard deviation from the training dataset, and spatial coordinates are normalized to the range $[0, 1]$ along each coordinate. For model architectures, FNO has a multi-channel input that involves the forcing term and normalized spatial coordinates, and its output is the corresponding PSF. We use a latent width of 8 in the lifting layer, 4 Fourier modes along each spatial dimension, and 2 Fourier layers. DeepONet takes the forcing term over the occupancy map as the branch input and normalized spatial coordinates as the trunk input, and the output is the corresponding PSF. Both the branch and trunk networks consist of two hidden layers of 128, followed by a latent output layer of width 128. The branch and trunk latent representations are combined through an inner product to reconstruct a scalar value at each grid point. We use GELU activation function and train a model for 500 epochs with a batch size of 16 using AdamW optimizer.

B. Performance of Neural Operators

We evaluate the approximation accuracy of the predicted PSFs, across in-distribution (ID) and out-of-distribution

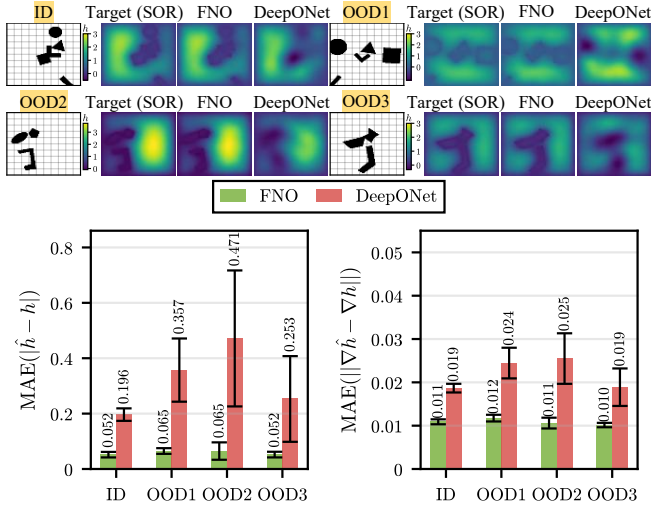


Fig. 3. Performance of learned neural operators. (Top) Given an occupancy map, we compute a target PSF using GPU-SOR and compare it with each PSF predicted by FNO and DeepONet, for an in-distribution (ID) case and three out-of-distribution (OOD) cases with variations in obstacle size, geometry, and configurations. FNO closely recovers the target PSF, whereas DeepONet shows substantially poorer approximation performance. (Bottom) Quantitative comparison of the approximation errors shows that FNO consistently achieves lower prediction errors (left) and gradient errors (right) than DeepONet across ID, OOD1, OOD2, and OOD3 settings.

(OOD) cases. As shown in the top-left occupancy map in Fig. 3, we consider a single ID case including five shapes with the same geometries and sizes as those in the training set, but with randomly generated positions and rotations. For three OOD settings, OOD1 includes unseen obstacle sizes while keeping the same shapes as those from the training set, whereas OOD2 and OOD3 involve unseen obstacle geometries. Figure 3 shows that FNO closely recovers the target PSF, as shown in heatmaps and achieves a mean approximation error of 0.058, which is significantly lower than that of DeepONet over ten randomly sampled occupancy maps for each case in the bottom-left. We further obtain the spatial gradients using finite differences and compare their approximation errors with those computed by the target PSFs. As shown in the bottom-right of Fig. 3, FNO also shows consistently lower gradient errors than DeepONet. The superior performance of FNO is likely attributed to its architecture that processes the full two-dimensional field and captures global spatial correlations between grids, whereas DeepONet operates on a concatenated representation that does not explicitly preserve the spatial correlations.

C. Robust Model-Free Safety-Critical Control

We demonstrate the proposed framework in collision avoidance scenarios on a mobile robot and a quadrotor. In these simulations, we use robot-centered local occupancy maps from simulated LiDAR range measurements of surrounding obstacles. We train neural operator models for each system with occupancy maps generated by uniformly sampling the poses of primitive obstacles (e.g. cylinders and cuboids) on a 2D grid (200^2) and a 3D grid (100^3), respectively. Both simulations are conducted in complex environments with moving obstacles whose geometries, sizes,

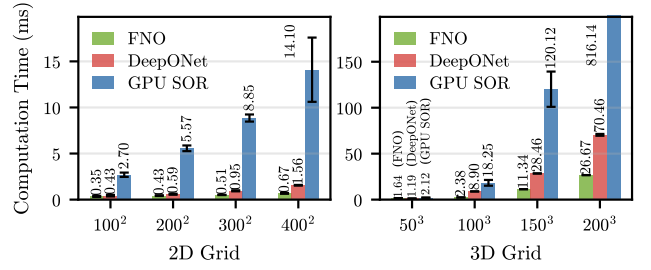


Fig. 4. Computation time of each method on mobile robot and quadrotor simulations as shown in Fig. 5. The GPU-parallel checkerboard numerical solver (GPU-SOR) built on CuPy shows efficient computation performance at a low grid resolution (100^2), however, its computational cost increases rapidly as the grid resolution increases. Compared to GPU-SOR, the neural operators maintain lower computation time at higher grid resolutions.

and configurations differ from those used in the training dataset. The task is to reach the goal position (orange) from the start position (blue) in the environment as shown in Fig. 5(a). We evaluate the proposed framework in terms of approximation errors and safety in this simulation. More examples (e.g. humanoid and quadruped) are available at <https://npsf-cbf.github.io>.

Mobile Robot We simulate a mobile robot with a unicycle kinematic model where the global robot's position is $\mathbf{y} = [x, y]^\top$, and the initial and goal positions are set to $[0.1, 0.1]^\top$ and $[7.0, 9.0]^\top$, respectively. A proportional goal tracking controller generates the nominal task-space velocity which is then transformed into the local frame of the robot. The proposed robust safety filter in (13) minimally modifies the nominal velocity to ensure safety using the estimated error bounds (18) from onboard LiDAR perception. The resulting safe velocity reference, $\dot{\mathbf{y}}_s = [v_x, v_y]^\top$ in the local frame is used to generate the unicycle reference: $v = \|\dot{\mathbf{y}}_s\|$ and $w = k_\theta \text{atan2}(v_y, v_x)$, $k_\theta > 0$. These commands are used as velocity references for the low-level velocity tracking controller. As shown in the first row of Fig. 5(b) and (c), both controllers with neural operators ensure safety, $h \geq 0$ in the complex dynamic environments. The mean approximation errors and their standard deviations over the field are $e_{\text{MAE}_h} = 0.010 \pm 0.003$, $e_{\text{MAE}_{\nabla h}} = 0.021 \pm 0.005$ for the FNO, and $e_{\text{MAE}_h} = 0.062 \pm 0.013$, $e_{\text{MAE}_{\nabla h}} = 0.143 \pm 0.022$ for DeepONet. We observe that the lower approximation error of the FNO enables its controller to closely reproduce the target safety function trajectory of GPU-SOR. In addition, the FNO-based controller yields smaller estimated errors than DeepONet as shown in Fig. 5(d). This can be interpreted as a consequence of more accurate predictions at the obstacle boundary, which result in $\hat{h}(\mathbf{y}^b, t) \approx 0$ and contribute to the smaller error estimates in (17).

Quadrotor We simulate a quadrotor with a 3D point-mass model, $m\ddot{\mathbf{y}} = \mathbf{F} - mge_z$, where $\mathbf{F} \in \mathbb{R}^3$ is the force input, $e_z = [0, 0, 1]^\top$, and m is the mass. $\mathbf{y} = [x, y, z]^\top$ is the robot's position in the global frame. The initial and goal positions are set to $[0.1, 0.1, 3.0]^\top$ and $[7.0, 9.0, 1.5]^\top$, respectively, as shown in Fig. 5(a). The nominal velocity generation and safety filtering follow the same procedure as in the mobile robot simulation. The resulting local safe velocity $\dot{\mathbf{y}}_s = [v_x, v_y, v_z]^\top$ is transformed into the global

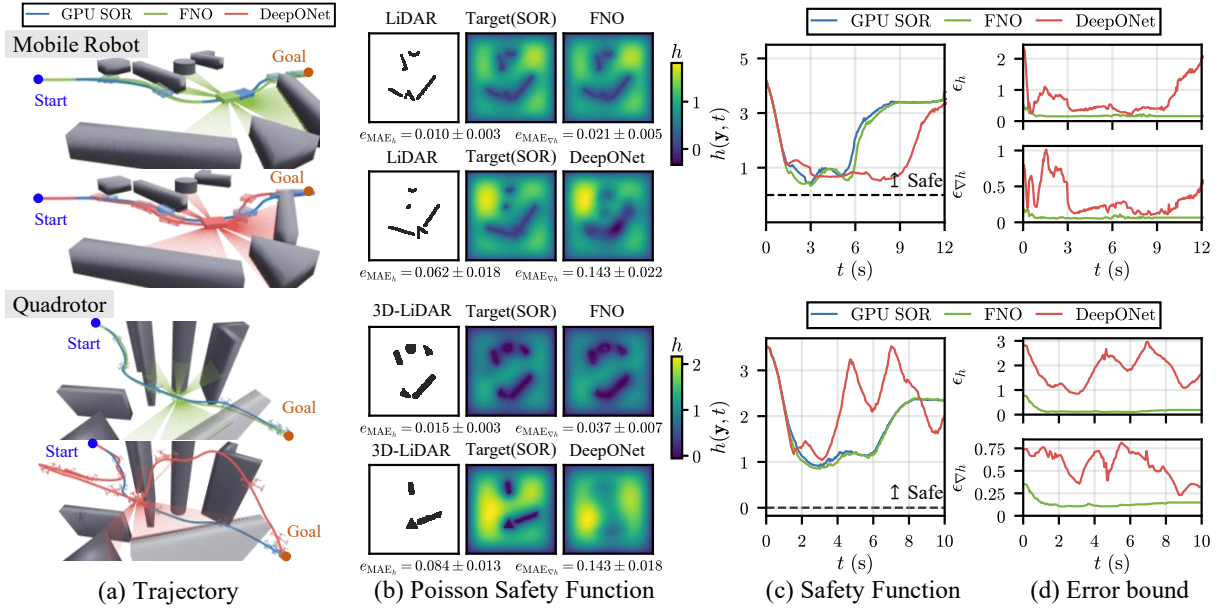


Fig. 5. Simulation results. (a) Both controllers with FNO and DeepONet achieve safe navigation between start (blue) and goal (orange) positions while satisfying $h(\mathbf{y}, t) \geq 0$ for a mobile robot in 2D (first row) and a quadrotor in 3D (second row). (b) Overall, the FNO achieves lower approximation errors of h and ∇h than DeepONet in both simulations. (c) The FNO-based controller closely matches the safety function profile to the target controller which uses a safety function computed from SOR. (d) It is observed that the lower approximation error leads to smaller error bound estimations.

frame to obtain $\dot{\mathbf{y}}_s^g$. The force input is then computed using a proportional velocity controller with gravity compensation, $\mathbf{F} = -K_d(\dot{\mathbf{y}} - \dot{\mathbf{y}}_s^g) + m\mathbf{g}e_z$. Rotational motion is restricted to yaw about the vertical axis, controlled by the commanded yaw rate in this simulation. As shown in the second row of Fig. 5(b) and (c), we observe that FNO predicts and recovers the target PSF in lower approximation error than DeepONet, while both neural operator-based controllers avoid collisions, maintaining $h \geq 0$. Consistent with 2D simulation results, the FNO-based controller also shows the lower error bounds estimation than using DeepONet as shown in the second row of Fig. 5(d). Overall, the DeepONet-based controller shows more conservative behavior than the FNO-based controller due to large approximation errors.

The computation times for each simulation at different grid resolutions are presented in Fig. 4. The performance of each method is evaluated on a desktop equipped with an NVIDIA RTX 4090 GPU. As shown in Fig. 4, the computation time of GPU-SOR to solve the PDE (3) substantially increases as the grid resolution increases, for example, 14.10 ms at 400^2 and 816.14 ms at 200^3 . By contrast, neural operators require less computation time at higher spatial resolutions ($\geq 200^2$). In particular, FNO predicts a PSF in 0.67 ms at 400^2 and 26.67 ms at 200^3 .

VII. EXPERIMENTAL VALIDATION

We demonstrate the proposed framework on MiR R100 commercial platform, which is equipped with an established low-level velocity controller, across four collision avoidance scenarios: static environment, moving obstacles with low and high robot speeds, as shown in Fig. 6. In the experiment, we use the same neural operator models without retraining as in the previous mobile robot simulation. The unicycle velocity references commanded to the actual robot are also computed

in the same manner. We control the robot in its local frame using local LiDAR measurements and set an offset radius of 0.50 m around the center of the robot to consider the robot's geometry. We adjust $\delta_h, \delta_{\nabla h}$ to achieve the desired conservatism, setting them to 0.03 and 0.01, respectively.

Results As shown in Fig. 6, the FNO-based controller maintains safety, $h \geq 0$ across all cases. In the Static and Moving 1 cases, both neural operator-based controllers provide positive safety function profiles, although DeepONet has a larger error bound estimation. In Moving 2, we change the box obstacles to a trash bin and a vacuum cleaner. DeepONet fails to properly represent the cleaner in the predicted PSF, and the run is terminated with an emergency stop. In contrast, the FNO-based controller continues to navigate while avoiding the small obstacle. In Moving fast case, we increase the robot speed, and both controllers show large variations in the safety function but still remain within the safe region. For all experiments, the mean absolute PSF approximation error at the obstacle boundary is 0.0058 for FNO and 0.0535 for DeepONet, with corresponding mean gradient errors of 0.178 and 0.325, respectively. The FNO-based controller overall yields smaller estimated error bounds than using DeepONet, although transient increases happen in the Moving Fast case. This is because the proposed error bound estimation relies on obstacle boundary observations, and insufficient measurements can lead to large error estimates. However, introducing an obstacle provides additional boundary observations, which can improve the local error estimates and reduce the conservatism of the safety constraint. For computation time, we measure computation time on a laptop equipped with an NVIDIA RTX 2060 GPU. For all experiments, the mean computation times ($\pm$ standard deviation) are 25.43 ± 5.61 ms for GPU-SOR, 2.79 ± 0.3 ms for FNO, and 3.54 ± 0.74 ms for DeepONet.

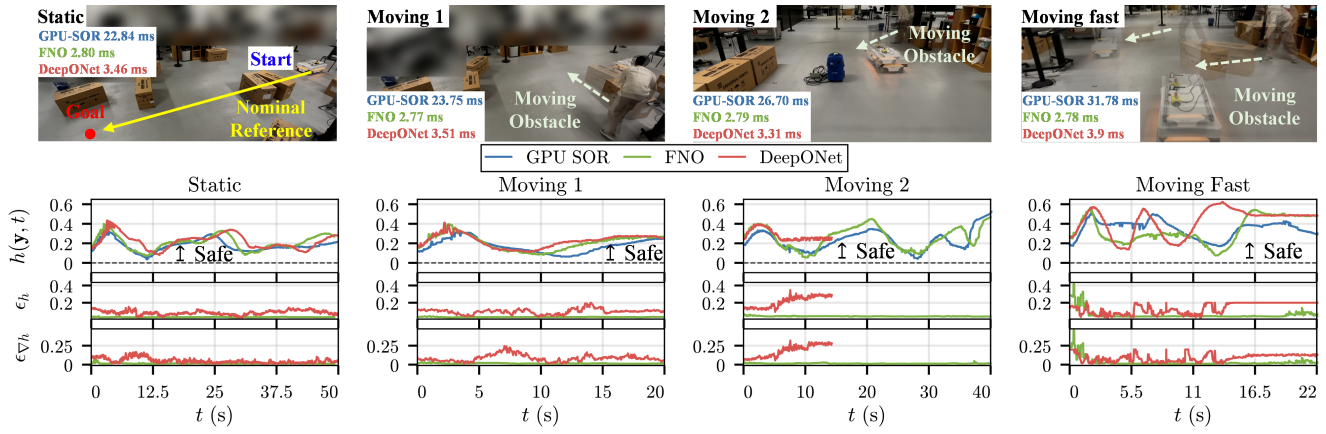


Fig. 6. Experimental evaluations of the proposed framework in several scenarios: static, moving obstacles, and increased robot speed (moving fast).

As a result, we validate the proposed framework, showing that it reduces computation time and achieves safe robot navigation. Notably, the FNO trained with only synthetic data enables safety-critical control using real-world sensing without retraining, and an additional calibration procedure may further enhance its prediction performance. In addition, $\delta_h, \delta_{\nabla h}$ can be selected based on estimation uncertainty, sensing noise, and the desired conservatism.

VIII. CONCLUSION

We propose a robust model-free safety-critical control framework that employs a neural operator to infer a PSF for safe robot navigation. The neural operator reduces computational cost by removing repeated PDE solves and can be efficiently incorporated into model-free safety-critical control in a worst-case robust manner. We further propose a perception-based local error bound estimation to facilitate practical implementation. We validate the proposed framework in both simulations and experiments, showing that FNO enables fast and accurate PSF prediction and is efficiently used in safety-critical control. A potential future research direction is to establish probabilistic safety guarantees by quantifying uncertainty in neural operator approximations.

REFERENCES

- [1] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *18th European Control Conference (ECC)*, 2019, pp. 3420–3431.
- [2] T. G. Molnar, R. K. Cosner, A. W. Singletary, W. Ubellacker, and A. D. Ames, "Model-free safety-critical control for robotic systems," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 944–951, 2022.
- [3] G. Bahati, R. M. Bena, and A. Ames, "Dynamic Safety in Complex Environments: Synthesizing Safety Filters with Poisson's Equation," in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [4] T. Chen and H. Chen, "Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems," *IEEE Transactions on Neural Networks*, vol. 6, no. 4, pp. 911–917, 1995.
- [5] Y. Kim, I. Iturrate, J. Langaa, and C. Sloth, "Safe robust adaptive control under both parametric and nonparametric uncertainty," *Advanced Robotics*, vol. 38, no. 5, pp. 357–366, 2024.
- [6] R. K. Cosner, A. W. Singletary, A. J. Taylor, T. G. Molnar, K. L. Bouman, and A. D. Ames, "Measurement-robust control barrier functions: Certainty in safety with uncertainty in state," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 6286–6291.
- [7] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. New York: John Wiley and Sons, 2005.
- [8] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *Proceedings. 1985 IEEE International Conference on Robotics and Automation*, vol. 2, 1985, pp. 500–505.
- [9] A. Singletary, K. Klingebiel, J. Bourne, A. Browning, P. Tokumaru, and A. Ames, "Comparative analysis of control barrier functions and artificial potential fields for obstacle avoidance," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 8129–8136.
- [10] H. Zhao, L. Brunke, O. Lagerquist, S. Zhou, and A. P. Schoellig, "Sq-cbf: Signed distance functions for numerically stable superquadric-based safety filtering," 2026. [Online]. Available: <https://arxiv.org/abs/2602.11049>
- [11] A. Singletary, W. Guffey, T. G. Molnar, R. Sinnet, and A. D. Ames, "Safety-critical manipulation for collision-free food preparation," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10954–10961, 2022.
- [12] D. Young, "Iterative methods for solving partial difference equations of elliptic type," *Transactions of the American Mathematical Society*, vol. 76, no. 1, pp. 92–111, 1954.
- [13] E. Yamaguchi, R. M. Bena, G. Bahati, and A. D. Ames, "Layered safety: Enhancing autonomous collision avoidance via multistage cbf safety filters," 2026. [Online]. Available: <https://arxiv.org/abs/2603.00338>
- [14] M. Wilkinson, G. Bahati, R. M. Bena, E. Fournay, J. W. Burdick, and A. D. Ames, "Full-body dynamic safety for robot manipulators: 3d poisson safety functions for cbf-based safety filters," 2026. [Online]. Available: <https://arxiv.org/abs/2604.21189>
- [15] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via deepnet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, vol. 3, no. 3, pp. 218–229, Mar 2021.
- [16] Z. Li, N. B. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," in *International Conference on Learning Representations*, 2021.
- [17] H. Hu and C. Liu, "Safe pde boundary control with neural operators," in *Proceedings of the 7th Annual Learning for Dynamics & Control Conference*, ser. Proceedings of Machine Learning Research, N. Ozay, L. Balzano, D. Panagou, and A. Abate, Eds., vol. 283. PMLR, 04–06 Jun 2025, pp. 513–526.
- [18] M. Lakshmidheepakreddy, S. Chen, and M. Fazlyab, "Domain adaptive safety filters via deep operator learning," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*, 2024, pp. 6389–6394.
- [19] S. Lanthaler, Z. Li, and A. M. Stuart, "Nonlocality and nonlinearity implies universality in operator learning," *Constructive Approximation*, vol. 62, no. 2, pp. 261–303, Oct 2025.
- [20] K. Long, C. Qian, J. Cortés, and N. Atanasov, "Learning barrier functions with memory for robust safe navigation," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4931–4938, 2021.
- [21] M. De Sa, P. Kotaru, and K. Sreenath, "Point cloud-based control barrier function regression for safe and efficient vision-based control," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 366–372.